Avaliação de Desempenho e Sobrecarga de Protocolo: TCP, UDP e QUIC com NGINX e Linux NetEm

João Marcos Sousa Rufino Leal[1]

[1]Departamento de Computação – Universidade Federal do Piauí (UFPI)
Bacharelado em Sistemas de Informação

jsousarufinoleal@ufpi.edu.br

Abstract. *This paper presents an experimental evaluation comparing TCP, pure UDP, and QUIC under controlled network conditions. Using Docker containers, NGINX, and Linux traffic control (tc/netem), we systematically evaluate goodput, flow completion time (FCT), jitter, transport overhead, and packet loss recovery across three distinct scenarios. The first scenario investigates lightweight UDP datagrams under increasing injection rates. The second scenario assesses large-scale HTTP transfers (100 MB and 1 GB) using HTTP/1.1, HTTP/2, and HTTP/3. The third scenario explores concurrent object requests in a high-latency channel subject to packet loss. Results indicate that pure UDP achieves minimal transport header overhead, TCP leverages kernel-space optimizations to sustain high throughput in stable channels, and QUIC mitigates head-of-line blocking while offering faster connection establishment via integrated TLS 1.3.*

Keywords: *TCP, UDP, QUIC, HTTP/3, TLS 1.3, Performance Evaluation, NetEm.*

Resumo. *Este trabalho apresenta uma avaliação experimental comparativa entre os protocolos TCP, UDP puro e QUIC sob condições controladas de rede. Utilizando contêineres Docker, NGINX e o subsistema de controle de tráfego do Linux (tc/netem), foram mensurados goodput, tempo de conclusão de fluxo (FCT), jitter, sobrecarga de cabeçalho e comportamento sob perdas de pacotes em três cenários planejados. O primeiro cenário analisa o transporte de pequenos datagramas UDP sob taxas crescentes de injeção. O segundo investiga transferências volumosas de 100 MB e 1 GB em HTTP/1.1, HTTP/2 e HTTP/3. O terceiro examina requisições concorrentes de objetos sob alta latência e descarte de pacotes. Os resultados demonstram que o UDP puro atinge a menor sobrecarga de transporte, o TCP sustenta alto throughput em canais estáveis por meio de otimizações no espaço de kernel, e o QUIC mitiga o bloqueio de cabeça de linha, além de acelerar o estabelecimento de conexão com TLS 1.3 integrado.*

Palavras-chave: *TCP, UDP, QUIC, HTTP/3, TLS 1.3, Avaliação de Desempenho, NetEm.*

Vídeo de Apresentação e Demonstração:
`https://youtu.be/Q8O8OcI9GoU`

1. Introdução e Objetivos

As aplicações de rede contemporâneas impõem exigências heterogêneas sobre a camada de transporte. Sistemas de telemetria e sensores industriais priorizam baixa latência e reduzida sobrecarga de cabeçalho, aceitando perdas eventuais. Em contraste, transferências de arquivos e distribuição de conteúdo em larga escala necessitam de entrega confiável, ordenação estrita e saturação eficiente da largura de banda disponível. Ao mesmo tempo, serviços web interativos demandam rapidez no estabelecimento de conexões seguras e resiliência a perdas transitórias em enlaces sem fio.

Historicamente, o protocolo TCP atendeu à maior parte do tráfego da Internet. Contudo, em enlaces de alta latência e sujeitos a ruído, o TCP exibe limitações intrínsecas decorrentes de seu aperto de mão em múltiplas etapas e do bloqueio de cabeça de linha (*Head-of-Line Blocking*). Nesse contexto, a padronização do protocolo QUIC pelo IETF buscou contornar esses gargalos operando sobre datagramas UDP, integrando nativamente a criptografia TLS 1.3 e viabilizando fluxos multiplexados independentes [Iyengar and Thomson 2021, Thomson and Turner 2021].

Este trabalho conduz uma investigação experimental comparativa entre três abordagens de transporte:

a) **TCP com TLS 1.3:** avaliado por meio dos protocolos HTTP/1.1 e HTTP/2 servidos pelo NGINX [Bishop 2022a];
b) **UDP Puro:** avaliado por meio de fluxos sintéticos de datagramas gerados pelo iperf3;
c) **QUIC:** avaliado por meio do protocolo HTTP/3 sobre UDP no NGINX [Bishop 2022b].

O objetivo central consiste em mensurar o comportamento de cada protocolo em condições controladas de rede, relacionando os resultados empíricos às decisões arquiteturais de cada pilha. A investigação organiza-se em três cenários planejados: o Cenário A favorece o UDP puro em baixa latência e alta taxa de envio; o Cenário B avalia transferências massivas em canal limpo; e o Cenário C examina o comportamento dos protocolos perante perdas aleatórias de pacotes e requisições concorrentes.

2. Fundamentação Teórica

Esta seção apresenta os conceitos necessários para interpretar os resultados experimentais. Primeiro, são discutidas as propriedades de confiabilidade e ordenação do TCP; em seguida, o baixo overhead e a ausência de garantias do UDP; por fim, a multiplexação, a segurança e o estabelecimento de conexão do QUIC.

2.1. TCP (Transmission Control Protocol) – RFC 793

O TCP provê um serviço de transporte orientado à conexão, com garantia de entrega ordenada e controle de integridade sobre um fluxo contínuo de bytes [Postel 1981]. O estabelecimento da comunicação ocorre por meio do aperto de mão de três vias (*three-way handshake*: SYN, SYN-ACK, ACK), no qual cliente e servidor sincronizam seus números de sequência iniciais e negociam parâmetros de canal.

A confiabilidade apoia-se em confirmações cumulativas (*Cumulative ACKs*) e temporizadores de retransmissão. Para evitar a sobrecarga do receptor, o TCP utiliza

o controle de fluxo baseado em janela deslizante. Já o controle de congestionamento da rede é gerenciado por algoritmos que alternam entre as fases de partida lenta (*Slow Start*), prevenção de congestionamento (*Congestion Avoidance*), retransmissão rápida (*Fast Retransmit*) e recuperação rápida (*Fast Recovery*).

Embora o HTTP/2 permita multiplexar várias requisições na mesma conexão TCP [Bishop 2022a], essa camada de transporte mantém uma abstração de fila única e sequencial. Caso um único segmento TCP seja descartado pelo canal, a camada de transporte retém os segmentos subsequentes até a retransmissão do pacote faltante, bloqueando a entrega de todos os fluxos multiplexados. Esse fenômeno denomina-se bloqueio de cabeça de linha no transporte.

2.2. UDP Puro (User Datagram Protocol) – RFC 768

O UDP é um protocolo de transporte simples, não orientado à conexão e sem garantias de entrega, confirmação ou preservação de ordem [Postel 1980]. Seu cabeçalho possui tamanho fixo de apenas 8 bytes, estruturado em quatro campos de 16 bits: Porta de Origem, Porta de Destino, Comprimento do Datagrama e Checksum.

A ausência de mecanismos de controle de congestionamento, controle de fluxo e temporizadores permite que o UDP transmita dados com atraso de enfileiramento mínimo e sem overhead de sincronização de estado. Essas características tornam o UDP adequado para tráfego em tempo real (como VoIP e transmissão de sensores), no qual a retransmissão de dados obsoletos é indesejada e o custo computacional por pacote deve ser mantido no menor nível possível.

2.3. QUIC (RFC 9000/9001) e TLS 1.3 (RFC 8446)

O QUIC é um protocolo de transporte multiplexado e seguro construído sobre o UDP no espaço de usuário [Iyengar and Thomson 2021]. O protocolo reestrutura o paradigma de transporte ao desacoplar a entrega de dados em fluxos (*streams*) independentes. Cada stream opera com controle de fluxo próprio. Dessa forma, a perda de pacotes de um stream não impede, no nível do transporte, o encaminhamento dos dados íntegros pertencentes a outros streams da mesma conexão, mitigando o bloqueio de cabeça de linha.

Diferentemente do TCP, no qual o número de sequência identifica a posição dos bytes no fluxo global, os números de pacote do QUIC avançam monotonamente dentro de cada espaço de números de pacote; dados retransmitidos são enviados em novos pacotes. Isso evita reutilizar números de pacote e contribui para a medição do RTT e a detecção de perdas. Adicionalmente, as conexões utilizam identificadores específicos (*Connection IDs*), que podem apoiar a continuidade de conexões durante mudanças de caminho.

A segurança do QUIC decorre da integração direta com o protocolo TLS 1.3 [Thomson and Turner 2021, Rescorla 2018]. Em vez de executar o aperto de mão criptográfico sobre uma conexão de transporte previamente estabelecida, o QUIC encapsula as mensagens de negociação do TLS 1.3 em seus pacotes iniciais. Essa unificação conclui a sincronização de chaves Diffie-Hellman e os parâmetros de transporte em apenas uma viagem de ida e volta (1-RTT), com suporte a envio imediato de dados em conexões prévias (0-RTT).

3. Metodologia Experimental e Topologia

A metodologia combina uma topologia isolada em contêineres, emulação controlada das condições do canal e repetição sistemática dos ensaios. Essa organização permite comparar os protocolos mantendo constantes a carga, o ambiente de execução e os critérios de medição.

3.1. Topologia e Ambiente Docker

Os experimentos foram conduzidos em um ambiente virtualizado isolado, composto por dois contêineres conectados via rede bridge Docker personalizada com sub-rede `172.28.0.0/16`. A Figura 1 ilustra a topologia do ambiente e a distribuição dos serviços.

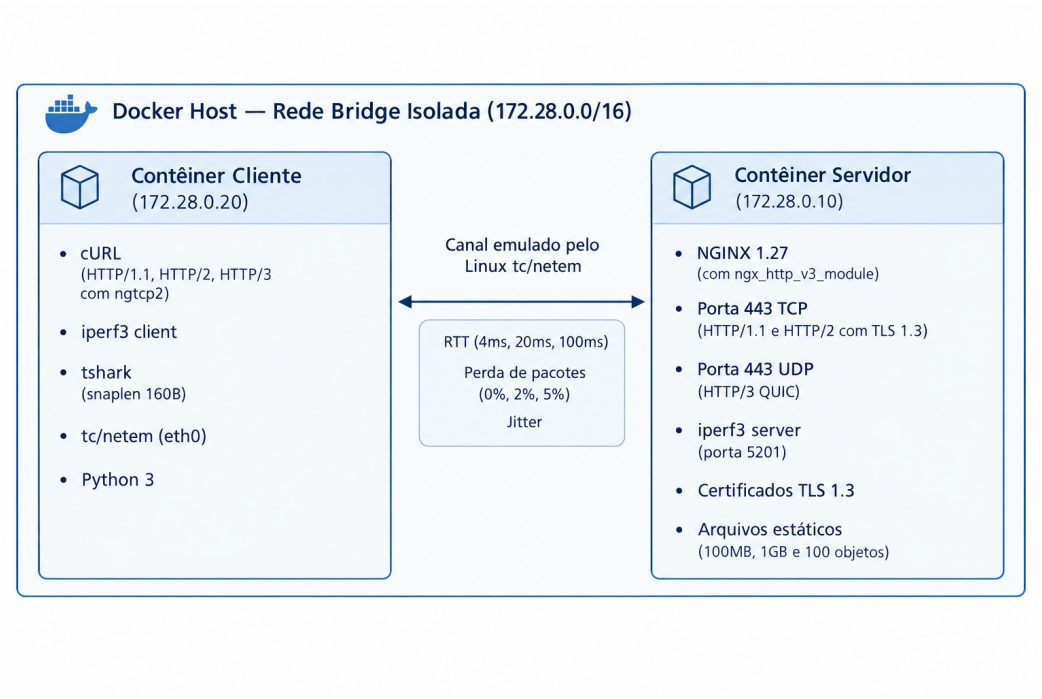


Figura 1. Topologia experimental baseada em contêineres Docker e emulação de canal.

O nó servidor (`172.28.0.10`) executa o NGINX 1.27 compilado com o módulo oficial `ngx_http_v3_module`. Esse serviço atende requisições HTTP/1.1 e HTTP/2 na porta 443 TCP, e requisições HTTP/3 na porta 443 UDP, com diretiva `reuseport` habilitada. Os certificados de teste foram gerados via OpenSSL com chave ECDSA na curva P-256 para uso com TLS 1.3. O contêiner servidor também disponibiliza o daemon `iperf3 -s` na porta 5201 UDP e armazena os artefatos estáticos de teste.

O nó cliente (`172.28.0.20`) possui o utilitário `curl` compilado com suporte nativo a HTTP/3 (utilizando as bibliotecas `ngtcp2` e `nghttp3`), cliente `iperf3`, ferramenta de captura `tshark` (Wireshark CLI), utilitários de rede `iproute2` e interpretador Python 3 com bibliotecas estatísticas.

3.2. Emulação de Canal e Parâmetros dos Cenários

A emulação das condições do enlace foi implementada na interface virtual `eth0` do contêiner cliente, utilizando o subsistema de controle de tráfego do kernel Linux (`tc`)

com o módulo `netem` (*Network Emulator*). A Tabela 1 resume a configuração aplicada a cada cenário.

Tabela 1. Parâmetros de emulação e cargas de trabalho por cenário experimental.

Cenário	Condições do Canal	Protocolos	Padrão de Carga	Ferramenta
Cenário A	RTT $\approx$ 4 ms, Perda 0%	UDP Puro	Datagramas 128 B (10–500 Mbps)	iperf3
Cenário B	RTT $\approx$ 20 ms, Perda 0%	H1.1, H2, H3	Arquivos de 100 MB e 1 GB	curl
Cenário C	RTT $\approx$ 100 ms, Perda 0%, 2%, 5%	H1.1, H2, H3	100 objetos concorrentes	curl

No Cenário A, aplicou-se atraso unidirecional de 2 ms (produzindo RTT de aproximadamente 4 ms) sem descarte, com injeção de taxas de 10, 50, 100, 250 e 500 Mbps em rajadas contínuas de 5 segundos. No Cenário B, configurou-se atraso de 10 ms (RTT de 20 ms) e perda nula para avaliar transferências de 100 MB e 1 GB. No Cenário C, configurou-se atraso de 50 ms (RTT de 100 ms) e perda aleatória de 0%, 2% e 5%, requisitando 100 arquivos independentes em paralelo.

3.3. Procedimento de Coleta e Rigor Estatístico

Para assegurar validade estatística, cada ponto experimental foi executado em 10 repetições independentes. As capturas de tráfego foram gravadas em formato PCAPNG via `tshark`, utilizando tamanho de captura truncado (*snaplen*) em 160 bytes para preservar todos os cabeçalhos de enlace, rede e transporte sem gerar arquivos excessivamente volumosos de carga útil.

A vazão útil (*goodput*) das transferências HTTP foi calculada pela relação entre o volume útil de dados recebidos com sucesso e o tempo total de transferência:

$$G = \frac{8 \cdot B}{T \cdot 10^6}, \tag{1}$$

em que B expressa o tamanho do recurso em bytes, T indica o tempo de conclusão de fluxo (FCT) em segundos e G representa o goodput em megabits por segundo (Mbps). O pipeline Python computou a média amostral (μ), o desvio padrão amostral (σ), os percentis p90 e p95, além do intervalo de confiança de 95% sob distribuição t-Student.

As baterias também passaram a registrar o tempo de CPU em espaço de usuário e em modo de sistema por execução, utilizando o utilitário `/usr/bin/time`. Esses valores são consolidados pelo pipeline em arquivos específicos de CPU para os três cenários. As capturas de tráfego dos Cenários B e C são produzidas separadamente por protocolo, evitando misturar HTTP/1.1, HTTP/2 e HTTP/3 na mesma amostra de overhead.

Como verificação adicional, foi incluído um teste de reconexões HTTP/3 com processos `curl` independentes. Esse teste mede novas conexões e seus tempos de estabelecimento, mas não é apresentado como prova de 0-RTT: a confirmação de envio antecipado de dados exige instrumentação específica do handshake ou um cliente que exponha esse evento.

4. Resultados Experimentais e Discussão

Os resultados são organizados de acordo com os três cenários definidos na metodologia. A análise começa pelo custo e pela vazão do UDP em baixa latência, passa pelas transferências volumosas em canal estável e termina com o comportamento dos protocolos sob perdas e requisições concorrentes.

4.1. Cenário A – Desempenho do UDP Puro em Baixa Latência

O Cenário A analisou o comportamento do UDP puro sob condições de latência baixa e ausência de perdas artificiais. As Figuras 2 e 3 apresentam, respectivamente, a relação entre a taxa de injeção e o goodput efetivo e a dispersão do jitter.

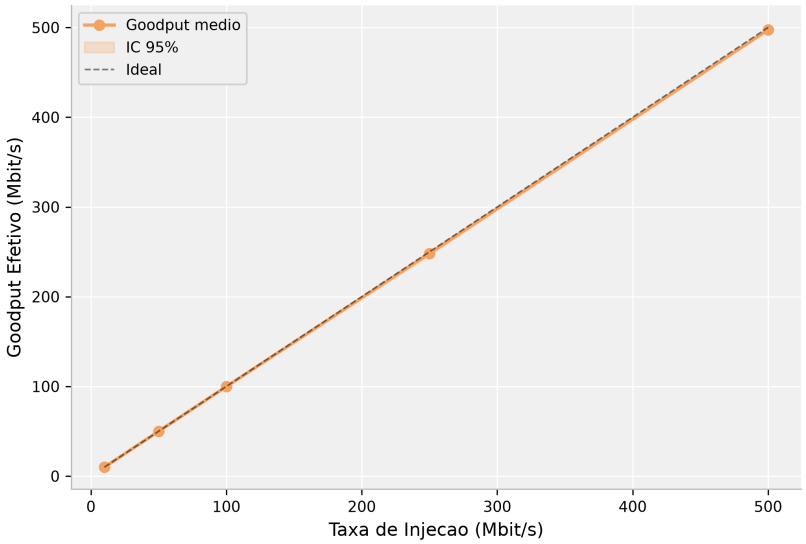


Figura 2. Vazão efetiva do UDP puro em função da taxa de injeção.

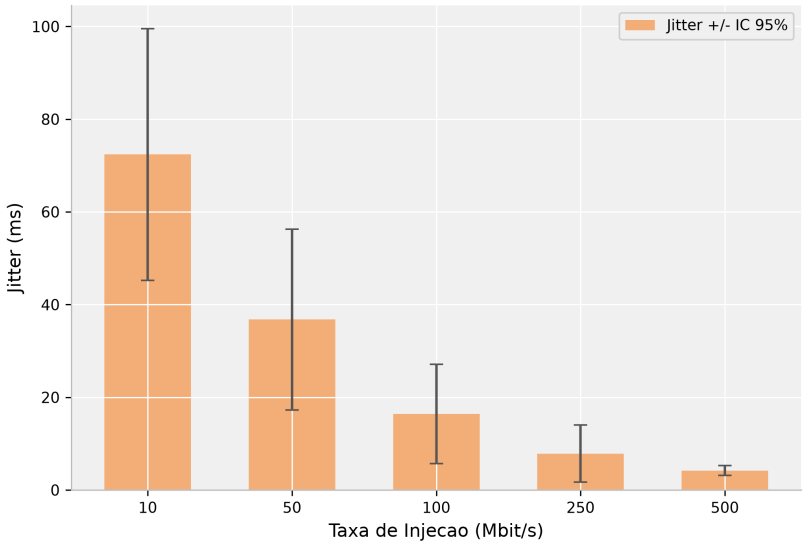


Figura 3. Jitter do UDP puro em função da taxa de injeção.

Nas taxas de 10, 50, 100 e 250 Mbps, o goodput acompanhou linearmente a taxa solicitada, atingindo médias de 9,99 Mbps, 49,95 Mbps, 99,93 Mbps e 249,73 Mbps, respectivamente. O jitter médio permaneceu abaixo de 0,13 ms em todos os ensaios. Ao atingir 500 Mbps, o goodput médio situou-se em $498{,}31 \pm 2{,}05$ Mbps. A coleta de CPU registrou também 3,50 s em modo de usuário e 1,55 s em modo de sistema nessa taxa; esses valores devem ser interpretados como tempo acumulado de CPU do processo medido.

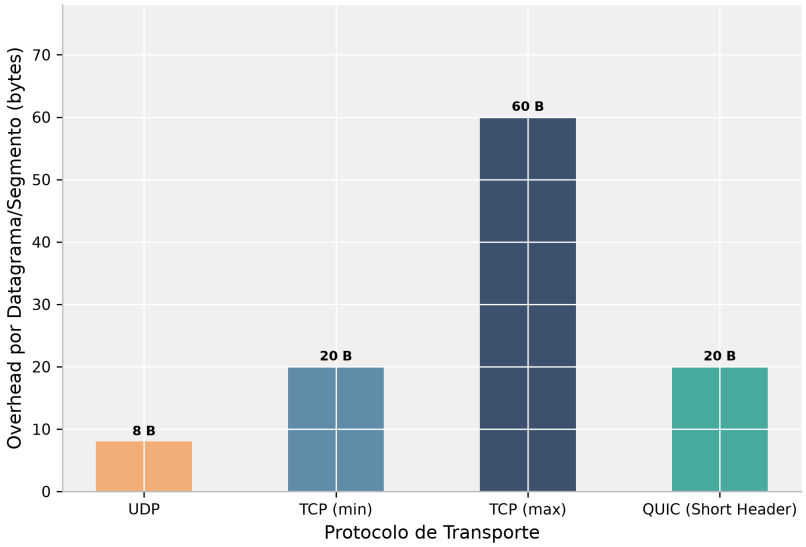


Figura 4. Comparação do overhead de cabeçalho de transporte entre UDP, TCP e QUIC.

A Figura 4 apresenta a sobrecarga teórica de cabeçalho de transporte. O UDP puro impõe custo fixo de apenas 8 bytes por datagrama. Em contrapartida, o cabeçalho base do TCP requer 20 bytes, alcançando 32 a 40 bytes quando acompanhado de opções comuns como confirmação seletiva (SACK) e marcas de tempo (*timestamps*). O QUIC introduz sobrecarga moderada em virtude do formato flexível de pacotes curtos e longos, além da proteção criptográfica. As capturas separadas por protocolo adicionadas ao pipeline permitem uma estimativa empírica complementar, mas o gráfico desta figura não representa a razão completa entre bytes brutos e bytes úteis da aplicação.

4.2. Cenário B – Eficiência do TCP em Transferências Massivas

O Cenário B comparou o desempenho das pilhas em transferências contínuas de 100 MB e 1 GB em canal estável (RTT de 20 ms, perda de 0%). A Tabela 2 consolida as medições obtidas para as três variantes de HTTP.

Tabela 2. Resultados consolidados do Cenário B (Média $\pm$ Desvio Padrão).

Arquivo	Protocolo	Handshake (ms)	TTFB (ms)	Goodput (Mbps)	FCT (s)
100 MB	HTTP/1.1	$27{,}12 \pm 4{,}47$	$48{,}14 \pm 4{,}81$	$561{,}19 \pm 64{,}67$	$1{,}52 \pm 0{,}20$
100 MB	HTTP/2	$25{,}02 \pm 0{,}99$	$40{,}65 \pm 1{,}11$	$594{,}13 \pm 50{,}69$	$1{,}42 \pm 0{,}13$
100 MB	HTTP/3	$16{,}78 \pm 2{,}33$	$33{,}59 \pm 3{,}23$	$45{,}95 \pm 0{,}27$	$18{,}26 \pm 0{,}11$
1 GB	HTTP/1.1	$24{,}69 \pm 0{,}76$	$49{,}23 \pm 8{,}23$	$587{,}77 \pm 31{,}70$	$14{,}66 \pm 0{,}85$
1 GB	HTTP/2	$25{,}12 \pm 1{,}26$	$41{,}34 \pm 2{,}31$	$596{,}64 \pm 39{,}53$	$14{,}46 \pm 1{,}07$
1 GB	HTTP/3	$16{,}88 \pm 2{,}06$	$36{,}31 \pm 5{,}78$	$45{,}84 \pm 0{,}26$	$187{,}38 \pm 1{,}07$

A Figura 5 ilustra o FCT e o goodput para ambos os arquivos. Em transferências volumosas, o TCP sobre HTTP/1.1 e HTTP/2 demonstrou superioridade de vazão líquida, sustentando taxas médias de 561,19 Mbps e 594,13 Mbps para 100 MB, e 587,77 Mbps e 596,64 Mbps para 1 GB, respectivamente. O HTTP/3 atingiu goodput estável em torno

de 46 Mbps, completando os arquivos de 100 MB e 1 GB em 18,26 e 187,38 segundos, respectivamente.

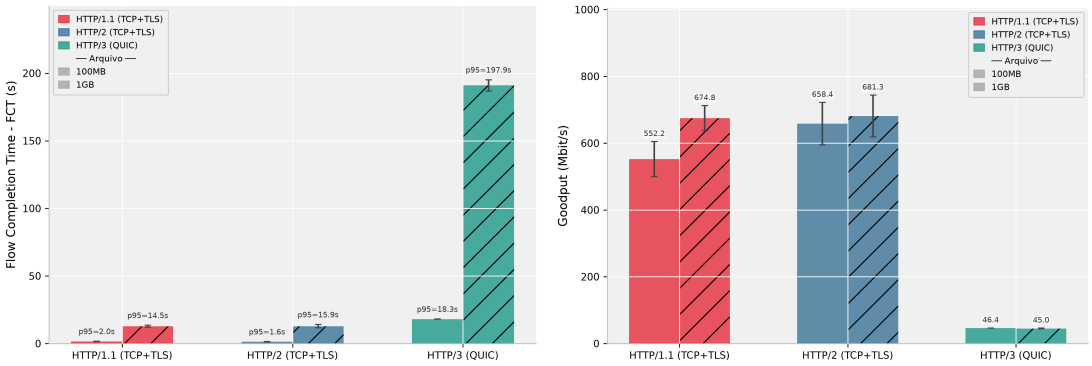


Figura 5. FCT e goodput para transferências de 100 MB e 1 GB em canal estável.

Essa diferença de taxa de transmissão reflete as características arquiteturais das implementações. O TCP é executado diretamente no núcleo do sistema operacional e se beneficia de otimizações de descarregamento de hardware consolidadas no Linux, como o *TCP Segmentation Offload* (TSO) e o *Generic Receive Offload* (GRO) [The Linux Kernel Documentation Project 2025]. Esses recursos agregam dados na camada de rede e reduzem drasticamente as trocas de contexto. Em contrapartida, a pilha QUIC utilizada opera em espaço de usuário, exigindo maior número de chamadas de sistema (*syscalls*) e processamento de criptografia datagrama a datagrama, o que limita a taxa máxima sustentável em canais de alta capacidade.

A coleta de CPU realizada durante as transferências de 100 MB e 1 GB registrou, para o arquivo de 1 GB, tempos médios de CPU de 1,62 s no HTTP/1.1, 1,89 s no HTTP/2 e 9,23 s no HTTP/3, calculados pela soma dos tempos de usuário e de sistema. Essa diferença é compatível com o maior custo de processamento da implementação QUIC neste ambiente. Como a métrica representa tempo de CPU, e não percentual de utilização normalizado pela duração da transferência, ela deve ser interpretada em conjunto com o FCT e o goodput.

Tabela 3. Tempo médio de CPU no Cenário B (usuário + sistema).

Arquivo	Protocolo	CPU (s)
100 MB	HTTP/1.1	0,15
100 MB	HTTP/2	0,17
100 MB	HTTP/3	0,86
1 GB	HTTP/1.1	1,62
1 GB	HTTP/2	1,89
1 GB	HTTP/3	9,23

Por outro lado, a Figura 6 evidencia a vantagem do QUIC no estabelecimento de conexão. Para transferências de 100 MB, o tempo medido pelo curl até a conclusão da negociação de aplicação e TLS (`time_appconnect`) foi de 16,78 ms no HTTP/3, contra 25,02 ms no HTTP/2 e 27,12 ms no HTTP/1.1. Em relação ao HTTP/2, essa diferença

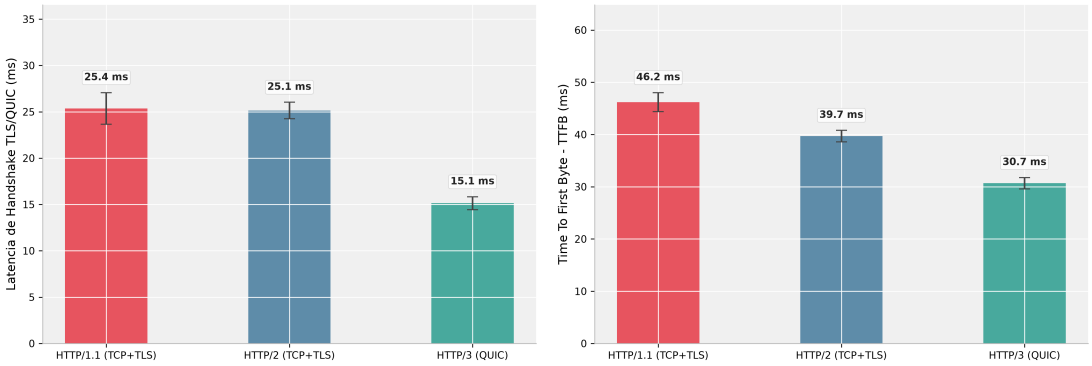


Figura 6. Comparação de latência de handshake e TTFB no Cenário B.

corresponde a uma redução de aproximadamente 33,0%. De forma correspondente, o TTFB do HTTP/3 foi 17,4% inferior ao do HTTP/2 (33,59 ms frente a 40,65 ms). Esses valores são compatíveis com o estabelecimento integrado do QUIC e do TLS 1.3, mas não constituem uma medição isolada de todos os componentes do handshake.

4.3. Cenário C – Comportamento dos Protocolos sob Perdas e Concorrência

O Cenário C avaliou o comportamento perante condições adversas de rede, combinando RTT de 100 ms com descarte aleatório de pacotes em 0%, 2% e 5% durante o download paralelo de 100 objetos independentes. A Tabela 4 consolida o FCT médio do lote, o percentil p95 e os eventos de retransmissão detectados pelo `tshark`.

Tabela 4. Resultados do Cenário C com 100 objetos concorrentes. O FCT médio é apresentado como média ± desvio padrão; o p95 é um valor pontual.

Perda (%)	Protocolo	FCT Médio (s)	FCT p95 (s)
0%	HTTP/1.1	$1{,}74 \pm 0{,}19$	1,96
0%	HTTP/2	$1{,}26 \pm 0{,}21$	1,52
0%	HTTP/3	$1{,}52 \pm 0{,}20$	1,85
2%	HTTP/1.1	$1{,}80 \pm 0{,}14$	1,98
2%	HTTP/2	$1{,}24 \pm 0{,}23$	1,63
2%	HTTP/3	$1{,}41 \pm 0{,}07$	1,51
5%	HTTP/1.1	$1{,}85 \pm 0{,}28$	2,30
5%	HTTP/2	$1{,}27 \pm 0{,}28$	1,71
5%	HTTP/3	$1{,}39 \pm 0{,}08$	1,51

A Figura 7 apresenta o FCT médio e o percentil p95 em função da taxa de descarte. No HTTP/1.1, a perda de 5% elevou o FCT médio para $1{,}85 \pm 0{,}28$ s com p95 de 2,30 s. O HTTP/2 manteve o menor FCT médio (1,27 s), enquanto o HTTP/3 apresentou FCT médio de 1,39 s e p95 de 1,51 s.

A Figura 8 destaca a função de distribuição cumulativa (CDF) do FCT dos lotes de 100 objetos sob 5% de perda. A independência dos streams no QUIC evita o bloqueio de cabeça de linha no transporte: a perda de dados em um stream não bloqueia, por si só, a entrega de dados já disponíveis em outros streams. Nesta nova execução, o HTTP/2

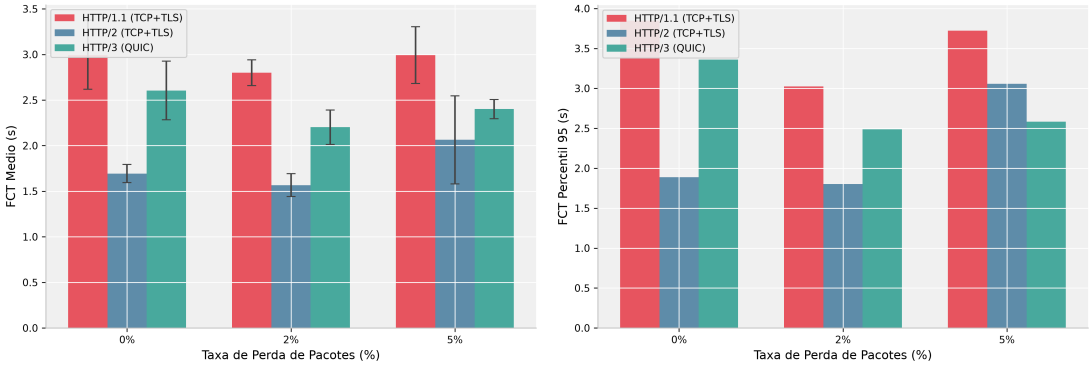


Figura 7. FCT médio e percentil p95 sob perdas de pacotes de 0%, 2% e 5%.

apresentou o menor FCT médio, mas o HTTP/3 apresentou p95 semelhante e menor que o HTTP/1.1. Assim, a vantagem arquitetural do QUIC não implicou automaticamente menor FCT médio, embora tenha reduzido a cauda em comparação com o HTTP/1.1 neste nível de perda.

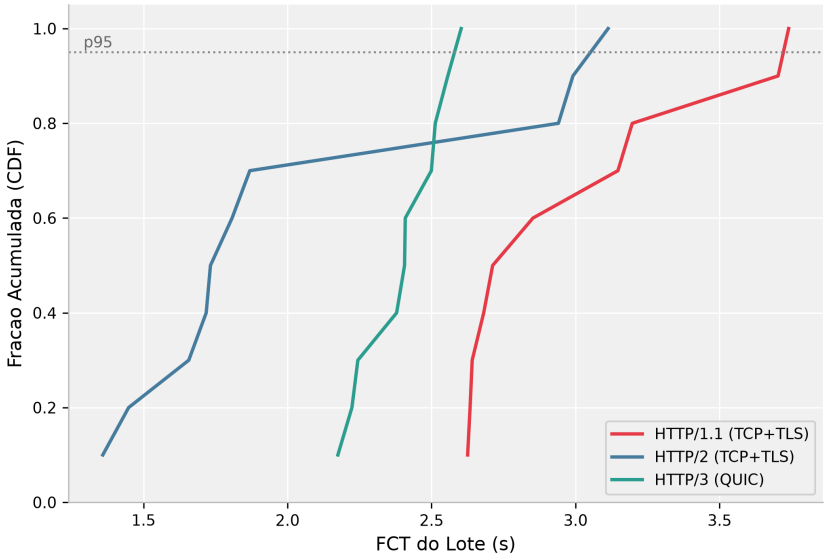


Figura 8. CDF do FCT dos lotes de 100 objetos sob 5% de perda.

A Figura 9 complementa a investigação com a análise das capturas em nível de enlace. Com a nova captura por protocolo, a análise passou a separar HTTP/1.1, HTTP/2 e HTTP/3. Nas capturas agregadas da execução anterior, foram identificados 142 segmentos TCP retransmitidos sob perda de 2% e 330 sob perda de 5%; as novas capturas permitem atribuir a contagem ao protocolo correspondente quando a bateria é processada novamente. A contagem não representa retransmissões QUIC: o pipeline contabiliza o campo de análise de retransmissão do TCP, portanto os eventos de recuperação do QUIC exigem análise específica de números de pacote, offsets e quadros ACK. O overhead calculado pelo script é uma estimativa de cabeçalhos sobre bytes capturados e deve ser interpretado separadamente da razão completa entre bytes brutos e bytes úteis da aplicação.

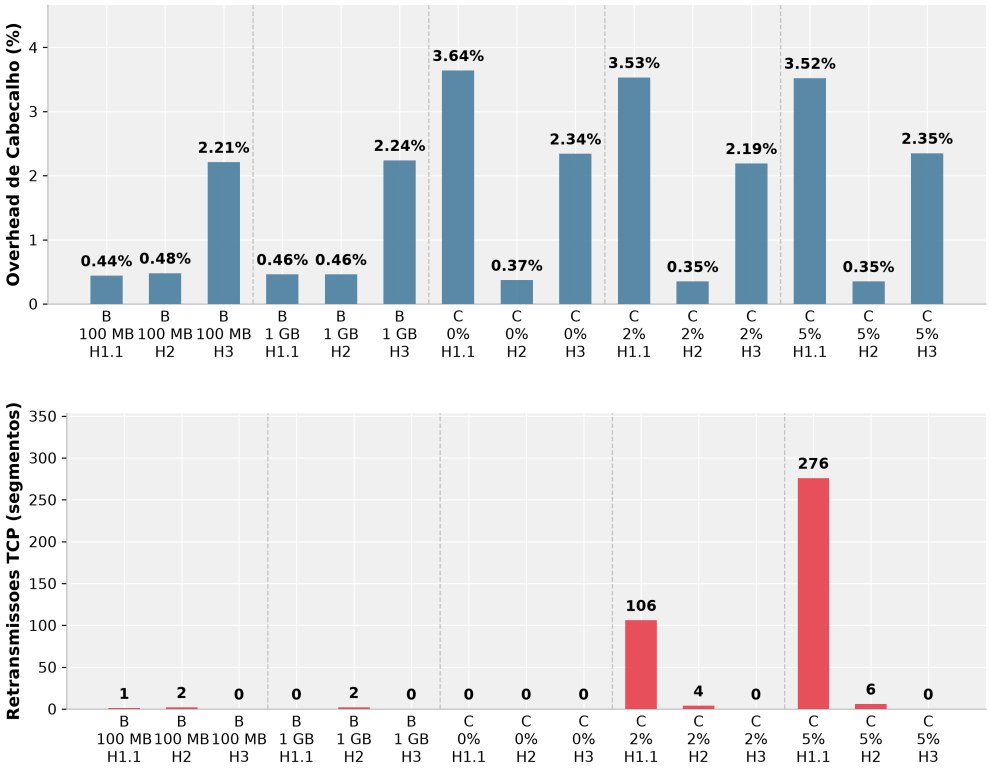


Figura 9. Análise de tráfego PCAP: overhead de protocolo (superior) e contagem de retransmissões TCP (inferior).

5. Considerações Finais e Conclusão

Este trabalho realizou uma avaliação experimental comparativa entre TCP, UDP puro e QUIC sob condições emuladas de rede com contêineres Docker e Linux NetEm. Os ensaios permitiram relacionar de forma empírica o desempenho de cada protocolo às suas decisões arquiteturais.

No Cenário A, o UDP puro confirmou sua aptidão para cenários de tempo real e baixa latência, sustentando transferências de até 500 Mbps com sobrecarga fixa de apenas 8 bytes por pacote e jitter médio contido em frações de milissegundo. Essa eficiência, todavia, prescinde de garantias de entrega e exige que a aplicação gerencie perdas quando necessário.

No Cenário B, o TCP sobre HTTP/1.1 e HTTP/2 demonstrou superioridade na vazão bruta para arquivos massivos, alcançando entre 561 e 597 Mbps, enquanto o HTTP/3 permaneceu próximo de 46 Mbps. Essa vantagem é compatível com a maturidade das otimizações de descarregamento no kernel Linux (TSO e GRO). Por sua vez, o QUIC (HTTP/3) destacou-se no tempo de estabelecimento de sessão, reduzindo o tempo medido por `time_appconnect` em aproximadamente 33,0% em relação ao HTTP/2.

No Cenário C, a avaliação confirmou a diferença arquitetural entre HTTP/2 sobre TCP e HTTP/3 sobre QUIC, mas mostrou que os efeitos dependem da métrica. Sob 5% de perda, o HTTP/2 apresentou o menor FCT médio (1,27 s), enquanto o HTTP/3 apresentou menor p95 que o HTTP/1.1 (1,51 s contra 2,30 s). Portanto, o resultado experimental sugere benefício do QUIC na cauda da distribuição, mas não superioridade geral no tempo

médio do lote.

Em síntese, os resultados evidenciam que inexiste um protocolo universalmente superior:

- O **UDP puro** é a escolha mais eficiente para fluxos contínuos de pequeno porte onde a simplicidade e a ausência de atraso de controle são prioritárias;
- O **TCP** permanece altamente eficaz para fluxos contínuos de grande porte em redes estáveis, aproveitando a aceleração nativa de kernel;
- O **QUIC** combina segurança integrada, rapidez no estabelecimento de conexões e independência entre streams; contudo, sua vantagem de desempenho sob perdas não foi confirmada pelo lote medido neste ambiente.

Como trabalhos futuros, sugere-se validar explicitamente o envio 0-RTT durante a retomada de sessões, comparar algoritmos modernos de controle de congestionamento no QUIC (como BBRv3 frente ao CUBIC), aprofundar a medição de ciclos de CPU com instrumentação eBPF e investigar o impacto de técnicas de offloading para UDP (como UDP GSO) na sobrecarga de processamento do QUIC em espaço de usuário.

Referências

Bishop, M. (2022a). HTTP/2. RFC 9113.

Bishop, M. (2022b). HTTP/3. RFC 9114.

Iyengar, J. and Thomson, M. (2021). QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000.

Postel, J. (1980). User Datagram Protocol. RFC 768.

Postel, J. (1981). Transmission Control Protocol. RFC 793.

Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446.

The Linux Kernel Documentation Project (2025). Segmentation Offloads. Linux kernel documentation.

Thomson, M. and Turner, S. (2021). Using TLS to Secure QUIC. RFC 9001.